Інтеграційне та end-to-end тестування

Тема 8

Як переконатися, що застосунок працює як ціле?

Рівні перевірки застосунку Spring Boot

- Модульні тести перевіряють логіку окремого класу із заглушками замість залежностей і не бачать конфігурації Spring.
- Тести-зрізи піднімають один шар (веб або доступ до даних) і перевіряють відображення JSON, валідацію та запити SQL.
- Інтеграційні тести запускають увесь контекст Spring із реальною базою даних у контейнері та замінюють лише зовнішні системи.
- End-to-end тести звертаються до запущеного застосунку так само, як клієнт: через HTTP або справжній браузер.

Зміст лекції

- Рівні тестів і контекст Spring у тестах
- Реальна база даних у Testcontainers
- Веб-шар, модулі та зовнішні сервіси
- End-to-end тести через HTTP і браузер
- Стабільність і швидкість тестового набору

Помилки, яких не бачать модульні тести

- Помилки відображення сутностей, запитів JPQL та обмежень таблиць виявляються лише на справжній базі даних.
- Схема бази даних, правила `SecurityFilterChain` і властивості `application.properties` не беруть участі в модульному тесті.
- Серіалізацію JSON, коди статусів і формат помилок визначають конвертери та обробники Spring MVC, а не код контролера.
- Межі транзакцій, події після фіксації та асинхронні слухачі працюють лише всередині контексту Spring.
- Тайм-аути й коди помилок зовнішнього HTTP-сервісу неможливо відтворити заглушкою Mockito.

Піраміда та трофей тестування

- Класична піраміда радить мати найбільше модульних тестів, менше інтеграційних і зовсім мало end-to-end.
- Модель Testing Trophy (Kent C. Dodds) ставить в основу статичний аналіз, а найбільшу частку відводить інтеграційним тестам, бо вони найкраще співвідносять вартість і впевненість.
- Для застосунків Spring Boot з базою даних і безпекою більшість дефектів виникає саме на стиках шарів, тому інтеграційних тестів зазвичай найбільше.
- End-to-end тести залишаються для кількох ключових сценаріїв користувача, оскільки вони найповільніші та найдорожчі в підтримці.

Межі тестів різних рівнів

Рівень	Що справжнє	Що замінено
Зріз (`@WebMvcTest`, `@DataJpaTest`)	один шар Spring	сусідні шари
Модуль (`@ApplicationModuleTest`)	біни модуля і база даних	інші модулі
Інтеграційний (`@SpringBootTest`)	увесь контекст і база даних	зовнішні HTTP-сервіси
End-to-end	застосунок на порту і клієнт	лише зовнішні системи

- Зріз запускається за сотні мілісекунд, повний контекст — за кілька секунд, а end-to-end тест через браузер триває секунди на кожен сценарій.

Модульний тест сервісу з Mockito

```
@ExtendWith(MockitoExtension.class)
class CameraServiceTest {

    @Mock CameraRepository repository;
    @Mock ApplicationEventPublisher eventPublisher;
    @InjectMocks CameraServiceImpl cameraService;

    @Test
    void registerCamera_WhenDuplicate_ShouldThrowAndNotPublish() {
        given(repository.existsByCameraId("CAM-01")).willReturn(true);

        assertThatThrownBy(() -> cameraService.registerCamera("CAM-01", "Вхід", "10.0.0.1"))
                .isInstanceOf(CameraAlreadyExistsException.class);

        then(repository).should(never()).save(any());
        then(eventPublisher).shouldHaveNoInteractions();
    }
}
```

Пояснення модульного тесту

- Розширення `MockitoExtension` створює заглушки `@Mock` і впроваджує їх у сервіс через конструктор за допомогою `@InjectMocks`.
- Тест не створює контекст Spring і виконується за мілісекунди, тому всі гілки бізнес-правил вигідно перевіряти саме так.
- Перевірки `then(...).should(...)` підтверджують, що при порушенні правила запис не збережено і подію не опубліковано.
- Без проксі Spring анотації `@Transactional` і `@PreAuthorize` не діють, тому транзакції та права перевіряють лише інтеграційні тести.

Тестові стартери Spring Boot 4.1

- Стартер `spring-boot-starter-test` містить JUnit Jupiter, Spring Test, AssertJ, Hamcrest, Mockito, JsonPath, JSONassert та Awaitility.
- У Spring Boot 4 тестова підтримка розділена за технологіями: `spring-boot-starter-webmvc-test`, `spring-boot-starter-data-jpa-test`, `spring-boot-starter-security-test`.
- Модуль `spring-boot-resttestclient` додає `RestTestClient` для тестів на справжньому порту.
- Модуль `spring-boot-testcontainers` разом з `testcontainers-postgresql` запускає базу даних у Docker.
- Бібліотеки `wiremock-standalone` та `playwright` підключаються окремо зі scope `test`.

Як @SpringBootTest піднімає застосунок

- Анотація шукає клас `@SpringBootConfiguration` вгору за пакетами тесту та створює повний контекст з автоконфігурацією.
- Застосовуються ті самі `application.properties`, профілі та ініціалізатори даних, що й під час звичайного запуску.
- Атрибут `properties` і `@TestPropertySource` перевизначають окремі властивості лише для тестів.
- Біни впроваджуються в тест через `@Autowired`, а розширення JUnit `SpringExtension` керує життєвим циклом контексту.

Режими webEnvironment

Режим	Сервер	Клієнт у тесті	Призначення
`MOCK` (за замовчуванням)	без сервера, імітація сервлетного середовища	`MockMvc`, `MockMvcTester`	швидкі інтеграційні тести веб-шару
`RANDOM_PORT`	вбудований Tomcat на вільному порту	`RestTestClient`, браузер	end-to-end тести
`DEFINED_PORT`	Tomcat на порту з `server.port`	будь-який	рідко, ризик конфлікту портів
`NONE`	без веб-середовища	виклики бінів	пакетна обробка, слухачі черг

Кеш контекстів Spring TestContext

- Створений контекст зберігається в кеші та повторно використовується всіма тестовими класами з такою самою конфігурацією.
- Ключ кешу складається з класів конфігурації, профілів, властивостей, імпортів і замінених бінів.
- Кеш живе в межах однієї JVM і за замовчуванням зберігає до 32 контекстів (`spring.test.context.cache.maxSize`).
- Статистику кешу видно в журналі на рівні DEBUG для категорії `org.springframework.test.context.cache` .

Що створює новий контекст

- Різний набір `@MockitoBean` або `@MockitoSpyBean` у тестових класах.
- Різні `@ActiveProfiles`, `@TestPropertySource`, атрибути `properties` чи `@DynamicPropertySource`.
- Різні класи в `@Import` або `@ContextConfiguration`.
- Анотація `@DirtiesContext`, яка знищує контекст після тесту або класу.
- Кожен новий контекст заново запускає Hibernate і власний контейнер бази даних, а в режимі `RANDOM_PORT` ще й Tomcat.

Спільна анотація для інтеграційних тестів

```
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@SpringBootTest(properties = "cctv.snapshots.dir=target/test-snapshots")
@AutoConfigureMockMvc
@Import({TestcontainersConfiguration.class, AlertsStubConfiguration.class})
@Sql(scripts = "/sql/reset-data.sql",
     executionPhase = Sql.ExecutionPhase.AFTER_TEST_METHOD)
public @interface IntegrationTest {
}
```

```
@IntegrationTest
class CameraRegistrationFlowTest { /* ... */ }
```

Пояснення спільної анотації

- Усі класи з `@IntegrationTest` мають однаковий ключ кешу, тому отримують один контекст і один контейнер бази даних.
- Тестові властивості задаються в анотації, а не через `@DynamicPropertySource` в окремих класах.
- Сценарій `@Sql` після кожного тесту видаляє створені записи й залишає демонстраційні камери з `CameraDataInitializer`, тому тести не залежать від порядку запуску.
- Замість заміни бінів через `@MockitoBean` зовнішні сервіси підміняються на рівні HTTP, що не ламає кеш.

Тестові зрізи Spring Boot

Анотація	Що створюється	Типове використання
`@WebMvcTest`	контролери, `@ControllerAdvice`, конвертери, фільтри, `WebMvcConfigurer`	статуси, валідація, JSON
`@DataJpaTest`	репозиторії, Hibernate, `TestEntityManager`	запити та обмеження таблиць
`@JdbcTest`	`DataSource`, `JdbcClient`, `JdbcTemplate`	SQL без JPA
`@RestClientTest`	`RestClient.Builder`, `MockRestServiceServer`	клієнти зовнішніх API
`@JsonTest`	налаштований Jackson	формат серіалізації

Зріз @WebMvcTest із MockMvcTester

```
@WebMvcTest(controllers = CameraApiController.class,
        excludeFilters = @ComponentScan.Filter(type = FilterType.ASSIGNABLE_TYPE,
                classes = JwtAuthenticationFilter.class))
@AutoConfigureMockMvc(addFilters = false)
class CameraApiControllerTest {

    @Autowired MockMvcTester mvc;
    @MockitoBean CameraService cameraService;

    @Test
    void addCamera_WithInvalidIp_ShouldReturn400() {
        assertThat(mvc.post().uri("/api/v1/cameras")
                .contentType(MediaType.APPLICATION_JSON)
                .content("""
                        { "cameraId": "CAM-10", "location": "Вхід", "ipAddress": "999" }
                        """))
                .hasStatus(HttpStatus.BAD_REQUEST);

        then(cameraService).shouldHaveNoInteractions();
    }
}
```

Пояснення зрізу веб-шару

- Клас `MockMvcTester` (Spring Framework 6.2+) поєднує MockMvc з AssertJ і не потребує `throws Exception` та статичних імпортів Hamcrest.
- Метод `hasStatus(...)` перевіряє код відповіді, а для тіла є `bodyJson().isLenientlyEqualTo(...)`, що порівнює JSON зі зразком у текстовому блоці без урахування порядку полів.
- Заглушка `@MockitoBean` замінює сервіс, тому тест перевіряє лише контролер, валідацію та обробник помилок.
- Параметр `addFilters = false` вимикає фільтри безпеки, а права перевіряються в інтеграційних тестах.

Пастки тестових зрізів

- Зріз `@WebMvcTest` підхоплює всі біни `Filter`, тому власний фільтр JWT з залежністю від сервісів зупиняє запуск контексту.
- Такий фільтр виключають атрибутом `excludeFilters` з типом `ASSIGNABLE_TYPE`, як у прикладі зрізу `CameraApiControllerTest`.
- Класи `@Configuration` зі `SecurityFilterChain` у зріз не потрапляють, і замість них діє стандартна конфігурація Spring Security.
- Кожен зріз з іншим набором `@MockitoBean` створює окремий контекст, тому кількість зрізів варто тримати під контролем.

Чому H2 не замінює PostgreSQL у тестах

- H2 у режимі сумісності імітує лише частину синтаксису PostgreSQL і не підтримує `jsonb`, частину функцій та розширень.
- Запити й типи стовпців, написані для PostgreSQL, на H2 доводиться спрощувати, тож перевіряється не та схема.
- Порядок сортування кирилиці, чутливість до регістру та поведінка блокувань відрізняються.
- Тест на H2 може пройти успішно там, де робоча база відхилить запит, і дати хибну впевненість.

Testcontainers і Docker

- Бібліотека Testcontainers через Docker API запускає контейнер з образу, чекає готовності сервісу та повертає його адресу і порт.
- Порт контейнера відображається на випадковий вільний порт хоста, тому паралельні збірки не конфліктують.
- Службовий контейнер Ryuk видаляє тестові контейнери після завершення JVM, навіть якщо тести перервано.
- Для запуску потрібен лише Docker, а встановлювати PostgreSQL на машину розробника чи сервер CI не треба.

Тести без доступного Docker

- На Windows Testcontainers працює з Docker Desktop на основі WSL 2 або з іншим сумісним рушієм, наприклад Rancher Desktop чи Podman з Docker API.
- Команда `docker info` має виконуватися без помилок ще до запуску тестів.
- Повідомлення `Could not find a valid Docker environment` означає, що Testcontainers не знайшов Docker, а не що тест чи конфігурація Spring помилкові.
- Анотація `@Testcontainers(disabledWithoutDocker = true)` позначає клас пропущеним замість упалого, якщо Docker недоступний.
- На сервері CI таку анотацію не використовують, бо пропущені тести дають успішну збірку без жодної перевірки бази.

Підключення контейнера через @ServiceConnection

```
@TestConfiguration(proxyBeanMethods = false)
public class TestcontainersConfiguration {

    @Bean
    @ServiceConnection
    PostgreSQLContainer postgres() {
        return new PostgreSQLContainer("postgres:17-alpine");
    }
}
```

```
@SpringBootTest
@Import(TestcontainersConfiguration.class)
class OrderServiceIntegrationTest { /* ... */ }
```

Пояснення @ServiceConnection

- Spring Boot створює бін `JdbcConnectionDetails` з адресою, логіном і паролем контейнера, тому властивості `spring.datasource.*` задавати не треба.
- Контейнер, оголошений як бін, запускається разом із контекстом і зупиняється разом із ним.
- Завдяки кешу контекстів один контейнер обслуговує всі тестові класи з однаковою конфігурацією.
- Версія образу має збігатися з версією бази даних у робочому середовищі.
- Для сервісів без `ConnectionDetails` властивості передає `@DynamicPropertySource` у тестовому класі або бін `DynamicPropertyRegistrar` у тестовій конфігурації.

Запуск застосунку з контейнерами під час розробки

```
public class TestApplication {

    public static void main(String[] args) {
        SpringApplication.from(Application::main)
                .with(TestcontainersConfiguration.class)
                .run(args);
    }
}
```

- Клас у `src/test/java` запускає застосунок з тією самою базою, що й тести: `.\mvnw.cmd spring-boot:test-run` або `.\gradlew.bat bootTestRun`.
- Альтернатива для звичайного запуску — залежність `spring-boot-docker-compose`, яка сама виконує `docker compose up` для `compose.yaml`.

Зріз @DataJpaTest з реальною базою

```
@DataJpaTest
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE)
@Import(TestcontainersConfiguration.class)
class CameraRepositoryTest {

    @Autowired CameraRepository repository;
    @Autowired TestEntityManager entityManager;

    @Test
    void saveAndFlush_DuplicateCameraId_ShouldViolateUniqueConstraint() {
        entityManager.persistAndFlush(new CameraEntity("CAM-20", "Вхід", "10.0.0.20", true));

        assertThatThrownBy(() -> repository.saveAndFlush(
                new CameraEntity("CAM-20", "Дублікат", "10.0.0.21", true)))
                .isInstanceOf(DataIntegrityViolationException.class);
    }
}
```

Пояснення зрізу доступу до даних

- Зріз `@DataJpaTest` створює лише репозиторії та Hibernate, тому запускається швидше за повний контекст.
- Компоненти на кшталт `CameraDataInitializer` у зріз не потрапляють, тому таблиця порожня і тест сам готує рядки через `TestEntityManager`.
- Значення `Replace.NONE` забороняє підміну джерела даних вбудованою базою і залишає PostgreSQL з контейнера.
- Кожен тест зрізу виконується в транзакції, яку після завершення відкочують.
- Обмеження таблиці перевіряється під час `INSERT`, який при генерації ключів через `SEQUENCE` відкладається до `flush`, тому `saveAndFlush` робить тест незалежним від стратегії генерації.

Перевірка таблиці, а не кешу Hibernate

```
@Test
void persistFlushFind_ShouldReadRowBackFromDatabase() {
    CameraEntity saved = entityManager.persistFlushFind(
            new CameraEntity("CAM-40", "Склад", "10.0.0.40", false));

    assertThat(saved.getId()).isNotNull();
    assertThat(saved.isActive()).isFalse();
}
```

```
String location = jdbc.sql("select location from cameras where camera_id = ?")
        .param("CAM-60")
        .query(String.class)
        .single();
assertThat(location).isEqualTo("Ворота");
```

MockMvc у повному контексті

- У режимі `MOCK` запит проходить через ланцюг фільтрів, `DispatcherServlet`, контролер, сервіс і базу без мережі.
- Тест бачить правила `SecurityFilterChain`, CSRF, валідацію, обробники помилок і шаблони Thymeleaf.
- Анотація `@WithMockUser` і постпроцесор `csrf()` задають користувача та токен, як у тестах попередньої лекції.
- Після запиту стан перевіряється безпосередньо в базі даних через `JdbcClient`, а не через той самий сервіс.

Повний сценарій реєстрації через форму

```
@IntegrationTest
@WithMockUser(roles = "ADMIN")
class CameraRegistrationFlowTest {

    @Autowired MockMvc mockMvc;
    @Autowired JdbcClient jdbc;

    @Test
    void createDuplicateViaForm_ShouldShowFieldError() throws Exception {
        mockMvc.perform(post("/cameras").with(csrf())
                        .param("cameraId", "CAM-01")
                        .param("location", "Дублікат")
                        .param("ipAddress", "10.0.0.1"))
                .andExpect(status().isOk())
                .andExpect(view().name("cameras/create-form"))
                .andExpect(model().attributeHasFieldErrorCode(
                        "form", "cameraId", "camera.id.duplicate"));
    }
}
```

Спільна анотація для end-to-end тестів

```
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT,
                properties = "cctv.snapshots.dir=target/test-snapshots")
@AutoConfigureRestTestClient
@Import({TestcontainersConfiguration.class, AlertsStubConfiguration.class})
@Sql(scripts = "/sql/reset-data.sql",
     executionPhase = Sql.ExecutionPhase.AFTER_TEST_METHOD)
public @interface E2ETest {
}
```

- Анотація відрізняється від `@IntegrationTest` режимом `RANDOM_PORT` і клієнтом `RestTestClient` замість MockMvc.
- Усі класи з `@E2ETest` (тести API і браузерні тести) отримують один контекст, один Tomcat і один контейнер бази даних.

RestTestClient на справжньому порту

```
@E2ETest
class ApiE2EIT {

    @Autowired RestTestClient client;

    @Test
    void viewerToken_ShouldNotCreateCamera() {
        String token = login("viewer", "viewer123");

        client.post().uri("/api/v1/cameras")
                .headers(headers -> headers.setBearerAuth(token))
                .contentType(MediaType.APPLICATION_JSON)
                .body("""
                        { "cameraId": "CAM-71", "location": "Дах", "ipAddress": "10.0.0.71" }
                        """)
                .exchange()
                .expectStatus().isForbidden();
    }
}
```

Пояснення RestTestClient

- Клас `RestTestClient` (Spring Framework 7) надсилає справжні HTTP-запити до застосунку на випадковому порту.
- Анотація `@AutoConfigureRestTestClient` налаштовує базову адресу `http://localhost:<порт>`.
- Допоміжний метод `login` надсилає POST на `/api/v1/auth/login` і повертає JWT з відповіді.
- Методи `expectStatus()`, `expectHeader()` і `expectBody().jsonPath(...)` перевіряють відповідь у плавному стилі.
- Тест і сервер працюють у різних потоках, тому `@Transactional` на тесті не відкочує зміни сервера.

Порівняння MockMvc і RestTestClient

Критерій	MockMvc	RestTestClient
Сервер	імітація сервлетного середовища	Tomcat на випадковому порту
Модель і представлення	доступні для перевірки	лише HTTP-відповідь
Куки, стиснення, перенаправлення	частково	як у справжнього клієнта
Типове використання	інтеграційні тести веб-шару	end-to-end тести API

Тестування модуля Spring Modulith

- Анотація `@ApplicationModuleTest` створює контекст лише з бінами пакета модуля та автоконфігурацією.
- Режим `STANDALONE` (за замовчуванням) не підключає інші модулі, а `DIRECT_DEPENDENCIES` і `ALL_DEPENDENCIES` додають їх.
- Тест перевіряє контракт модуля: публічний API, опубліковані події та реакцію на події інших модулів.
- Залежності від бінів інших модулів у режимі `STANDALONE` замінюються через `@MockitoBean`.
- Виклик `ApplicationModules.of(CctvApplication.class).verify()` без запуску контексту перевіряє, що між модулями немає циклів і ніхто не звертається до внутрішніх пакетів чужого модуля.

Сценарій із публікацією події

```
@ApplicationModuleTest
@Import(TestcontainersConfiguration.class)
@Sql(scripts = "/sql/reset-data.sql",
     executionPhase = Sql.ExecutionPhase.AFTER_TEST_METHOD)
@WithMockUser(roles = "ADMIN")
class CameraModuleTest {

    @Autowired CameraService cameraService;

    @Test
    void registerCamera_ShouldPublishEvent(Scenario scenario) {
        scenario.stimulate(() -> cameraService.registerCamera("CAM-50", "Ворота", "10.0.0.50"))
                .andWaitForEventOfType(CameraRegisteredEvent.class)
                .matching(event -> event.cameraId().equals("CAM-50"))
                .toArriveAndVerify(event -> assertThat(event.location()).isEqualTo("Ворота"));
    }
}
```

Пояснення API Scenario

- Метод `stimulate` виконує дію в транзакції, яку фіксують, тому події після фіксації доставляються слухачам.
- Зафіксовані дані залишаються в таблиці, тому тест модуля теж прибирає їх сценарієм `@Sql`.
- Метод `publish` надсилає подію в модуль так, ніби її опублікував інший модуль.
- Методи `andWaitForEventOfType` і `andWaitForStateChange` чекають на результат з тайм-аутом замість фіксованої паузи.
- Параметр `AssertablePublishedEvents` дає змогу перевірити, що подію не було опубліковано.

Асинхронні слухачі та Awaitility

- Після `@EnableAsync` слухачі `@ApplicationModuleListener` виконуються в окремому потоці, і запит завершується раніше за них.
- Виклик `Thread.sleep` або сповільнює тести, або дає випадкові падіння на повільному сервері CI.
- Бібліотека Awaitility повторює перевірку з короткими інтервалами, доки вона не пройде або не вичерпається тайм-аут.

```
await().atMost(Duration.ofSeconds(5)).untilAsserted(() ->
        alertsServer.verify(postRequestedFor(urlEqualTo("/alerts"))
                .withRequestBody(containing("CAM-60"))));
```

Зовнішні HTTP-сервіси в тестах

- Заглушка `@MockitoBean` для клієнта не виконує HTTP-запит і створює окремий контекст.
- Зріз `@RestClientTest` з `MockRestServiceServer` перехоплює запити всередині `RestClient` без мережі.
- WireMock запускає справжній HTTP-сервер на випадковому порту, і застосунок надсилає запити так само, як у робочому середовищі.
- Контрактні тести (Spring Cloud Contract, Pact) додатково перевіряють, що заглушка відповідає реальному сервісу.

Заглушка WireMock як бін тестового контексту

```
@TestConfiguration(proxyBeanMethods = false)
public class AlertsStubConfiguration {

    @Bean(destroyMethod = "stop")
    WireMockServer alertsServer() {
        WireMockServer server = new WireMockServer(options().dynamicPort());
        server.start();
        server.stubFor(post("/alerts").willReturn(ok()));
        return server;
    }

    @Bean
    DynamicPropertyRegistrar alertsProperties(WireMockServer alertsServer) {
        return registry -> registry.add("cctv.alerts.url",
                () -> alertsServer.baseUrl() + "/alerts");
    }
}
```

Перевірка запиту, помилок і тайм-аутів

```
alertsServer.verify(1, postRequestedFor(urlEqualTo("/alerts"))
        .withHeader("Content-Type", containing("application/json"))
        .withRequestBody(matchingJsonPath("$.title", equalTo("CAMERA_ADDED"))));
```

```
alertsServer.stubFor(post("/alerts").willReturn(serverError()));
assertThatThrownBy(() -> webhookClient.send("TEST", "Перевірка"))
        .isInstanceOf(HttpServerErrorException.class);

alertsServer.stubFor(post("/alerts").willReturn(ok().withFixedDelay(1_500)));
assertThatThrownBy(() -> webhookClient.send("TEST", "Перевірка"))
        .isInstanceOf(ResourceAccessException.class);
```

- Робочий тайм-аут клієнта становить 2 секунди, тому тестовий клас знижує його через `@TestPropertySource(properties = "cctv.alerts.timeout=500ms")`, і затримка заглушки в 1,5 секунди його перевищує.

Переваги WireMock над заглушкою клієнта

- Перевіряється реальний запит: URL, метод, заголовки і тіло JSON після серіалізації.
- Коди помилок 4xx і 5xx, повільні відповіді та розірвані з'єднання відтворюються налаштуванням заглушки.
- Тайм-аути, повторні спроби та обробка помилок клієнта перевіряються без доступу до зовнішнього сервісу.
- Бін `DynamicPropertyRegistrar` передає адресу заглушки без `@DynamicPropertySource`, тому кеш контексту зберігається.

Що додає end-to-end тест

- Браузер виконує JavaScript, зберігає куки, переходить за перенаправленнями та застосовує заголовки CSP.
- Перевіряється взаємодія клієнта SPA із сервером: маршрутизація, кука `XSRF-TOKEN` і заголовок `X-XSRF-TOKEN`.
- Тест описує сценарій користувача й тому виявляє прогалини між окремо перевіреними частинами.
- Приклад такої прогалини: кнопку створення приховано від ролі `VIEWER`, але сторінка форми відкривається за прямою адресою.

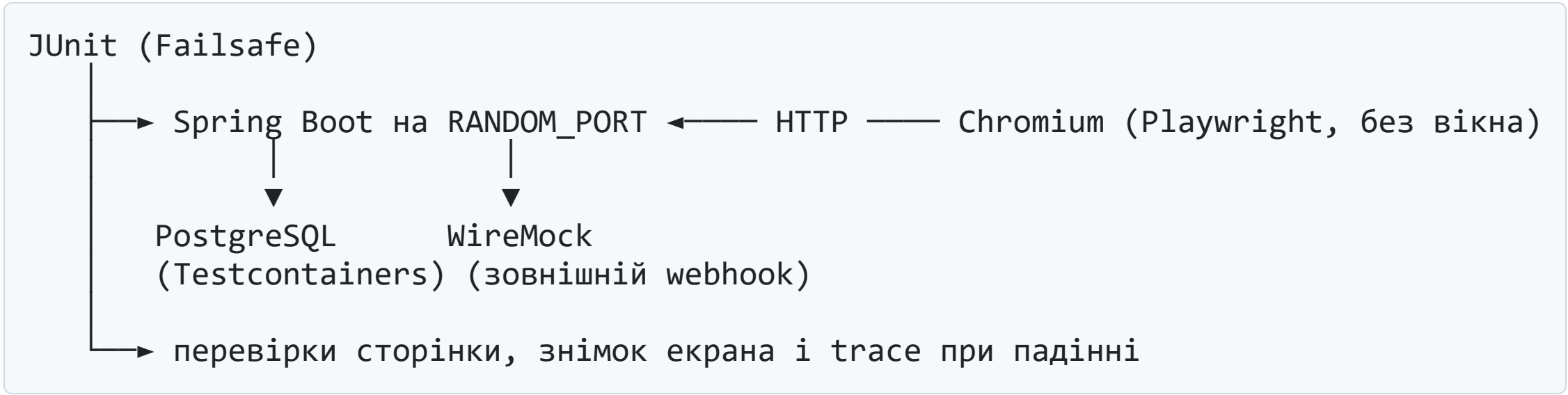
Інструменти для браузерних тестів

Інструмент	Мови	Очікування елементів	Особливості
Playwright	Java, TypeScript, Python, .NET	автоматичне	три рушії браузерів, trace
Selenium WebDriver	Java та інші	явне (`WebDriverWait`)	стандарт W3C, багато наявних тестів
Cypress	JavaScript, TypeScript	автоматичне	працює всередині браузера

Вибір Playwright для проєкту на Java

- Playwright з 2024 року найшвидше зростає серед інструментів браузерного тестування, а Selenium залишається найпоширенішим у наявних проєктах.
- Офіційна бібліотека `com.microsoft.playwright` для Java підключається як звичайна залежність Maven і працює з JUnit.
- Автоматичне очікування елементів прибирає більшість причин нестабільності, характерних для тестів Selenium.
- Однаковий API підходить і для сторінок Thymeleaf, і для односторінкових клієнтів.

Архітектура браузерного тесту



Підготовка браузера та контексту

```
abstract class BrowserTestSupport {

    static Playwright playwright;
    static Browser browser;

    @LocalServerPort int port;
    BrowserContext context;
    Page page;

    @BeforeAll
    static void launchBrowser() {
        playwright = Playwright.create();
        browser = playwright.chromium().launch();
    }

    @BeforeEach
    void openPage() {
        context = browser.newContext(new Browser.NewContextOptions()
                .setBaseURL("http://localhost:" + port));
        page = context.newPage();
    }

    @AfterEach void closePage() { context.close(); }
    @AfterAll static void closeBrowser() { playwright.close(); }
}
```

Пояснення життєвого циклу браузера

- Один процес браузера запускається на клас тестів, бо його старт займає секунди.
- Кожен тест отримує новий `BrowserContext` зі своїми куками та сесією, як у нового користувача.
- Базова адреса з `@LocalServerPort` дає змогу писати відносні шляхи `page.navigate("/login")`.
- Під час першого запуску Playwright завантажує браузер у кеш користувача, а на сервері CI це робиться окремою командою.

Локатори за ролями та автоматичне очікування

- Метод `getByRole(AriaRole.BUTTON, ...)` знаходить елемент за роллю й доступною назвою, як його бачить користувач.
- Методи `getByLabel` і `getByText` шукають поля за підписом і елементи за текстом.
- Селектори CSS та XPath прив'язують тест до розмітки і ламаються при зміні верстки.
- Перевірки `assertThat(locator).isVisible()` чекають на виконання умови до 5 секунд без `Thread.sleep`.
- Атрибути доступності (`label for`, `role="alert"`) одночасно покращують інтерфейс і спрощують тести.

Браузерний тест форми Thymeleaf

```
@E2ETest
class ThymeleafUiE2EIT extends BrowserTestSupport {

    @Test
    void admin_ShouldRegisterCameraThroughForm() {
        page.navigate("/login");
        page.getByLabel("Ім’я користувача").fill("admin");
        page.getByLabel("Пароль").fill("admin123");
        page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Увійти")).click();

        page.getByRole(AriaRole.LINK, new Page.GetByRoleOptions().setName("Додати камеру")).click();
        page.getByLabel("ID камери").fill("CAM-77");
        page.getByLabel("Локація").fill("Підвал");
        page.getByLabel("IP-адреса").fill("10.0.0.77");
        page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Зареєструвати")).click();

        assertThat(page.getByRole(AriaRole.ALERT)).containsText("CAM-77");
    }
}
```

Page Object для сторінок

```
final class CameraPages {

    private final Page page;

    CameraPages(Page page) { this.page = page; }

    Locator addCameraLink() {
        return page.getByRole(AriaRole.LINK, new Page.GetByRoleOptions().setName("Додати камеру"));
    }

    Locator row(String cameraId) {
        return page.getByRole(AriaRole.ROW).filter(new Locator.FilterOptions().setHasText(cameraId));
    }

    Locator alert() { return page.getByRole(AriaRole.ALERT); }

    void registerCamera(String cameraId, String location, String ipAddress) {
        addCameraLink().click();
        page.getByLabel("ID камери").fill(cameraId);
        page.getByLabel("Локація").fill(location);
        page.getByLabel("IP-адреса").fill(ipAddress);
        page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Зареєструвати")).click();
    }
}
```

Браузерний тест клієнта SPA

```
@E2ETest
class ReactUiE2EIT extends BrowserTestSupport {

    @Test
    void cameraCreatedInSpa_ShouldBeVisibleInBothInterfaces() {
        loginAs("admin", "admin123");
        page.navigate("/app/");
        CameraPages cameras = new CameraPages(page);

        cameras.registerCamera("CAM-78", "Горище", "10.0.0.78");

        assertThat(cameras.alert()).hasText("Камеру CAM-78 успішно підключено.");
        page.navigate("/cameras");
        assertThat(cameras.row("CAM-78")).containsText("10.0.0.78");
    }

    @Test
    void anonymousUser_ShouldBeRedirectedToLogin() {
        page.navigate("/app/");
        assertThat(page).hasURL(Pattern.compile(".*/login$"));
    }
}
```

Пояснення тестів інтерфейсу

- Page Object збирає локатори та дії зі сторінкою в одному класі, а тест описує лише сценарій.
- Однакові підписи полів і кнопок дають змогу використати той самий Page Object для Thymeleaf і SPA.
- Метод `loginAs` у `BrowserTestSupport` виконує ті самі кроки входу через `/login`, що й тест форми Thymeleaf, а створена сесія діє і для клієнта SPA.
- Тест SPA перевіряє всю ланку: вхід через сесію, отримання токена CSRF, POST з JavaScript і відображення відповіді.
- Збірка клієнта SPA має виконатися до запуску тестів, тому браузерні тести запускаються на етапі `verify`.

Діагностика падінь через trace і знімки екрана

```
context.tracing().start(new Tracing.StartOptions()
        .setScreenshots(true)
        .setSnapshots(true));
```

```
page.screenshot(new Page.ScreenshotOptions().setPath(Paths.get("target/playwright/failed.png")));
context.tracing().stop(new Tracing.StopOptions().setPath(Paths.get("target/playwright/trace.zip")));
```

- Trace містить кожну дію, знімок DOM до і після неї, мережеві запити та журнал консолі.
- Файл відкривається командою `npx playwright show-trace trace.zip` або на сайті trace.playwright.dev.
- Артефакти зберігаються лише для тестів, що впали, через розширення JUnit `AfterTestExecutionCallback`.

Окремий запуск end-to-end тестів

- Плагін Maven Surefire запускає класи `*Test` на етапі `test`, а плагін Failsafe запускає класи `*IT` на етапі `integration-test`.
- Команда `.\mvnw.cmd test` дає швидкий зворотний зв'язок, а `.\mvnw.cmd verify` додає end-to-end тести.
- У Gradle завдання `test` виключає класи `*E2EIT`, а окреме завдання `e2eTest` запускає лише їх.
- Команда `.\gradlew.bat test` запускає швидкі тести, а `.\gradlew.bat check` додає end-to-end тести, бо завдання `check` залежить від `e2eTest`.

Завдання Gradle для end-to-end тестів

```
tasks.named('test') {
    useJUnitPlatform()
    exclude '**/*E2EIT.class'
}

tasks.register('e2eTest', Test) {
    group = 'verification'
    useJUnitPlatform()
    testClassesDirs = sourceSets.test.output.classesDirs
    classpath = sourceSets.test.runtimeClasspath
    include '**/*E2EIT.class'
    shouldRunAfter tasks.named('test')
}

tasks.named('check') {
    dependsOn tasks.named('e2eTest')
}
```

End-to-end тести на сервері CI

- Серверу CI потрібні Docker і браузер Chromium разом із системними бібліотеками.
- У Maven браузер встановлює команда `./mvnw exec:java -D exec.mainClass=com.microsoft.playwright.CLI -D exec.args="install --with-deps chromium"`.
- У Gradle те саме робить завдання `./gradlew installPlaywright` типу `JavaExec`, яке запускає той самий клас `com.microsoft.playwright.CLI`.
- Браузер працює без вікна (headless), а звіти `target/failsafe-reports` для Maven, `build/reports/tests` для Gradle і тека `target/playwright` зберігаються як артефакти збірки.

Нестабільні тести та їхні причини

- Фіксовані паузи `Thread.sleep` замість очікування умови.
- Спільні дані між тестами та залежність від порядку запуску.
- Залежність від поточного часу, часового поясу, мови системи або випадкових значень без фіксованого зерна.
- Фіксовані порти та звернення до реальних зовнішніх сервісів.
- Асинхронна обробка без очікування результату та перевірки, що спираються на розташування елементів сторінки.

Керований час через бін Clock

```
@Test
void getCameras_WithExpiredToken_ShouldReturnProblemDetail401() throws Exception {
    Clock halfHourAgo = Clock.offset(Clock.systemUTC(), Duration.ofMinutes(-30));
    String expiredToken = new JwtService(jwtSecret, 15, halfHourAgo).generateToken(viewer);

    mockMvc.perform(get("/api/v1/cameras").header(AUTHORIZATION, "Bearer " + expiredToken))
            .andExpect(status().isUnauthorized());
}
```

- Код з `Instant.now()` чи `new Date()` не дає перевірити прострочення, тому `JwtService` і `JwtValidatorService` отримують бін `Clock` через конструктор.
- У модульному тесті `Clock.fixed(...)` задає точний момент, і перевірка за хвилину до та через хвилину після закінчення строку стає детермінованою.
- Інтеграційний тест створює прострочений токен окремим екземпляром сервісу, тому бін `Clock` у контексті не замінюється і кеш зберігається.

Швидкість тестового набору

- Мінімальна кількість різних конфігурацій контексту дає мінімальну кількість стартів Spring і контейнерів.
- Зрізи замість повного контексту там, де перевіряється один шар.
- Контейнер бази даних один на контекст, а не на кожен тестовий клас.
- Кілька end-to-end тестів для ключових сценаріїв замість перевірки кожного поля через браузер.
- Поточний час виконання та найповільніші класи видно у звітах Surefire і Failsafe.

Тести в нативному образі GraalVM

- Команди `.\mvnw.cmd -PnativeTest test` і `.\gradlew.bat nativeTest` компілюють тести разом із застосунком у нативний образ і запускають їх без JVM.
- Для обох збірок потрібні GraalVM і плагін Native Build Tools: `native-maven-plugin` у Maven та `org.graalvm.buildtools.native` у Gradle.
- Такий запуск виявляє відсутні підказки рефлексії та ресурси, наприклад для об'єктів у шаблонах Thymeleaf чи JSON.
- Контекст Spring для тестів обробляється AOT заздалегідь, тому `@MockitoBean` та інші динамічні заміни бінів не підтримуються.
- Нативні тести довго збираються, тому їх запускають на CI для гілки релізу, а не під час щоденної розробки.

Типові помилки інтеграційних тестів

- Використання H2 замість бази даних робочого середовища.
- Різні `@MockitoBean` і властивості в кожному класі, через що на кожен клас запускається новий контекст і контейнер.
- `@Transactional` на тесті, який перевіряє події після фіксації, асинхронну обробку або HTTP-запит до сервера на порту.
- Перевірка результату через той самий сервіс, що його записав, замість читання таблиці.
- Тести, що проходять лише в певному порядку через дані, залишені іншими тестами.

Типові помилки end-to-end тестів

- Пошук елементів за CSS-класами та структурою DOM замість ролей і підписів.
- Повторення в браузері всіх перевірок валідації, які вже покрито інтеграційними тестами.
- Спільний `BrowserContext` для кількох тестів, через що сесія одного тесту впливає на інший.
- Відсутність знімків екрана і trace, через що падіння на сервері CI неможливо відтворити.
- Запуск end-to-end тестів разом із швидкими тестами на кожну зміну коду.

Висновки

- Інтеграційні тести з повним контекстом Spring і реальною базою даних виявляють більшість дефектів на стиках шарів.
- Testcontainers з `@ServiceConnection` замінює H2 справжнім PostgreSQL, а спільна конфігурація тестів зберігає один контекст і один контейнер.
- Зрізи, тести модулів і заглушки WireMock дають змогу перевіряти окремі частини швидко та без зовнішніх систем.
- End-to-end тести через `RestTestClient` і Playwright перевіряють сценарії користувача так, як їх бачить клієнт.
- Стабільний набір тестів не залежить від порядку, часу та пауз і очікує на результат замість фіксованих затримок.

Індивідуальне завдання

1. Взяти проєкт Spring Boot з REST API, який виконує CRUD-операції над сутністю через Spring Data JPA.
2. Додати тестові залежності `spring-boot-testcontainers` і `testcontainers-postgresql` та драйвер `postgresql`, а робочу конфігурацію застосунку залишити без змін.
3. Створити в `src/test/java` клас `TestcontainersConfiguration` з біном `PostgreSQLContainer` і анотацією `@ServiceConnection`, щоб тести працювали з PostgreSQL у Docker.

Індивідуальне завдання

1. Написати інтеграційний тест з `@SpringBootTest`, `@AutoConfigureMockMvc` і `@Import(TestcontainersConfiguration.class)`, який через MockMvc від імені користувача з `@WithMockUser` створює запис POST-запитом і через `JdbcClient` перевіряє, що рядок з'явився в таблиці.
2. Написати другий тест у тому самому класі, який надсилає некоректні дані й перевіряє статус 400 та незмінну кількість рядків у таблиці.
3. Запустити `.\mvnw.cmd test` або `.\gradlew.bat test` і завантажити архів проєкту без `target` і `build` та знімок екрана з результатом тестів.

Групове завдання

1. Додати в командний проєкт тестові залежності Testcontainers і клас `TestcontainersConfiguration` з `@ServiceConnection`, щоб тести працювали з PostgreSQL у Docker.
2. Створити спільну анотацію з `@SpringBootTest`, `@AutoConfigureMockMvc` і `@Import(TestcontainersConfiguration.class)`, щоб усі тестові класи використовували один контекст Spring і один контейнер.
3. Покрити інтеграційними тестами через MockMvc CRUD-операції основних сутностей проєкту і перевіряти результат безпосередньо в таблицях через `JdbcClient`.

Групове завдання

1. Покрити інтеграційними тестами ключовий сценарій предметної області від HTTP-запиту до таблиці, зокрема права різних ролей.
2. Якщо проєкт звертається до зовнішніх HTTP-сервісів, замінити їх у тестах заглушками WireMock і перевірити обробку помилок.
3. Написати end-to-end тести REST API через `RestTestClient` для основного сценарію проєкту.
4. Написати браузерні тести Playwright для двох-трьох ключових сценаріїв інтерфейсу з попередньої лекції, використовуючи Page Object.

Групове завдання

1. Відокремити end-to-end тести від швидких тестів (Failsafe або окреме завдання Gradle).
2. Завантажити звіт зі структурою тестів за рівнями, часом виконання, результатом запуску `.\mvnw.cmd verify` або `.\gradlew.bat check` і знімками екрана або trace браузерних тестів.
3. Завантажити архів проєкту без `target` і `build`.